WAREED

Blood Donation Network

Software Requirements Specification (SRS)

Version	1.0
Status	Draft
Date	18 April 2026
Platform	Web (Laravel + PWA)
Real-Time	Laravel Reverb (WebSockets)
Location	City/District-Based

Connecting Hospitals & Donors — Saving Lives in Real Time

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for Wareed, a hospital-centered real-time blood donation network. The platform connects blood donors with hospitals in emergency situations, automating the matching and notification process to reduce response time and save lives.

1.2 Project Overview

What is Wareed?

Wareed (وريد — Arabic for 'vein') is a web-based platform built as a Progressive Web App (PWA).

It bridges the gap between hospitals with urgent blood needs and registered donors in matching locations.

Instead of donors searching for cases, the system proactively finds and notifies the right donors automatically.

The platform targets the Egyptian market as its primary audience, with a scalable architecture for regional expansion.

1.3 Scope

- Hospitals create and manage emergency blood requests through a dedicated portal.
- The system automatically matches donors based on blood type and city/district.
- Matched donors receive instant push notifications via WebSockets (Laravel Reverb).
- Donors can accept or decline requests; hospitals see responses in real time.
- The platform is installable on mobile devices as a PWA (no app store required).
- An Admin panel manages users, verifications, and platform health.

1.4 Definitions & Acronyms

Term	Definition
SRS	Software Requirements Specification
PWA	Progressive Web App — a web app installable on mobile devices
Reverb	Laravel Reverb — self-hosted WebSocket server
Echo	Laravel Echo — JavaScript library for real-time event listening
ERD	Entity Relationship Diagram
Blood Type	ABO + Rh factor (e.g. A+, O−, AB+, B−)

Emergency Request	A hospital's urgent need for a specific blood type
Matching	Finding donors whose blood type is compatible with the request
Notification	Real-time push alert sent to matched donors instantly

1.5 Tech Stack Summary

Layer	Technology	Purpose
Backend	Laravel 11 (PHP)	API, business logic, authentication, matching engine
Real-Time	Laravel Reverb + Echo	WebSocket server for instant push notifications
Frontend	Blade + Alpine.js / Livewire	UI rendering and lightweight interactivity
Database	MySQL	Primary relational data store
Auth	Laravel Sanctum / Breeze	Session and token-based authentication
PWA	Web App Manifest + Service Worker	Installable app experience on mobile
Notifications	Laravel Notifications + WebSockets	In-app + push alerts to donors
Hosting	Any VPS (e.g. DigitalOcean)	Self-hosted Reverb, nginx, PHP-FPM

2. Stakeholders & System Actors

2.1 Actor Overview

Wareed has three distinct user types. Each has a separate registration flow, dashboard, and set of permissions.

Actor	Description	Access Level
Donor	Registered individual willing to donate blood	Donor Dashboard
Hospital	Verified medical institution creating blood requests	Hospital Dashboard
Admin	Platform operator managing users and system health	Admin Panel (full access)
Guest	Unauthenticated visitor — can view public info only	Public pages only

2.2 Donor

- Registers with personal info: name, phone, blood type, city/district, date of last donation.
- Completes profile to become 'available' in the matching pool.
- Receives real-time push notifications when a matching blood request is created.
- Accepts or declines requests from the notification or dashboard.
- Views personal donation history and earned trust badges.
- Can set themselves as temporarily unavailable (e.g. recently donated).

2.3 Hospital

- Registers and waits for Admin verification (prevents fake accounts).
- Creates emergency blood requests specifying blood type, quantity, and urgency level.
- Views real-time status of each request: how many donors were notified, who responded.
- Marks a request as fulfilled when blood has been secured.
- Manages their profile and contact information.

2.4 Admin

- Approves or rejects hospital registration requests.
- Views all donors, hospitals, and active requests on a central dashboard.
- Can suspend or deactivate accounts that violate platform rules.
- Monitors system statistics: total requests, fulfillment rate, active donors by city.

3. Functional Requirements

3.1 Authentication & Registration

3.1.1 Donor Registration

1. The system shall allow any user to register as a Donor.
2. Required fields: Full name, Email, Password, Phone number, Blood type (A+/A−/B+/B−/AB+/AB−/O+/O−), City, District, Date of birth.
3. Optional fields: Date of last donation (used to calculate eligibility cooldown).
4. Email verification must be completed before the donor account becomes active.
5. Donors are automatically placed in the matching pool once verified.

3.1.2 Hospital Registration

6. The system shall allow medical institutions to register as Hospitals.
7. Required fields: Hospital name, Official email, Password, Phone, City, District, License/registration number.
8. After registration, hospital status is set to 'Pending' until approved by Admin.
9. Hospital receives an email notification when their account is approved or rejected.
10. Hospitals cannot create blood requests until their account is verified.

3.1.3 Login & Sessions

11. All user types log in with email and password.
12. The system shall redirect each user type to their respective dashboard after login.
13. Password reset via email link must be supported.
14. Sessions shall expire after a configurable period of inactivity.

3.2 Emergency Blood Request Management

3.2.1 Creating a Request

15. Only verified hospitals can create blood requests.
16. Required fields: Blood type needed, Quantity (number of units), Urgency level (Critical / Urgent / Standard), Contact person name and phone, Additional notes (optional).
17. Upon creation, the request status is set to 'Active' and the matching engine is triggered immediately.
18. Each request is assigned a unique ID and timestamp.

3.2.2 Request Lifecycle

Status	Meaning	Who Changes It

Active	Request is open and donors are being notified	System (on creation)
Partially Fulfilled	Some donors have responded but more are needed	System (auto-update)
Fulfilled	Hospital has confirmed the blood need is met	Hospital
Expired	Request was not fulfilled within the set time window	System (auto)
Cancelled	Hospital manually cancelled the request	Hospital

3.2.3 Managing Requests

19. Hospitals can view all their requests with current status in their dashboard.
20. Hospitals can mark a request as Fulfilled or Cancelled at any time while Active.
21. Hospitals can see the list of donors who accepted each request, with their contact details.
22. Hospitals cannot edit an active request — they must cancel and recreate if details change.

3.3 Donor Matching Engine

3.3.1 Matching Rules

Blood Type Compatibility Chart

O− can donate to: O−, O+, A−, A+, B−, B+, AB−, AB+ (Universal Donor)
O+ can donate to: O+, A+, B+, AB+
A− can donate to: A−, A+, AB−, AB+
A+ can donate to: A+, AB+
B− can donate to: B−, B+, AB−, AB+
B+ can donate to: B+, AB+
AB− can donate to: AB−, AB+
AB+ can donate to: AB+ (Universal Receiver)

23. The matching engine shall find all donors who: (a) have a compatible blood type for the request, AND (b) are located in the same city as the requesting hospital, AND (c) have status 'Available', AND (d) are past their donation cooldown period (default: 56 days / 8 weeks).
24. Results are sorted by: district match first (same district as hospital), then by last donation date (longest ago first).
25. The system shall notify all matching donors simultaneously — not one at a time.

26. If no donors are found in the same district, the system expands to the full city automatically.

3.3.2 Donor Availability

27. Donors can set their status to: Available, Temporarily Unavailable, or On Cooldown (system-managed).
28. When a donor logs a successful donation, the system automatically sets a 56-day cooldown.
29. Donors on cooldown are excluded from the matching pool until cooldown expires.

3.4 Real-Time Notification System

3.4.1 Donor Notifications

30. When a blood request is created, all matched donors receive an instant push notification via Laravel Reverb WebSocket.
31. The notification shall display: Hospital name, Blood type needed, Urgency level, City/District, Hospital contact number.
32. Donors receive the notification whether they are currently in the app or have the PWA installed in the background.
33. Donors can Accept or Decline the request directly from the notification or their dashboard.

3.4.2 Hospital Real-Time Updates

34. Hospitals see their active request dashboard update live without page refresh.
35. When a donor accepts a request, the hospital dashboard instantly shows the donor's name and contact number.
36. A live counter shows: Notified / Accepted / Declined for each request.

3.4.3 Notification Log

37. All notifications are stored in the database with timestamps and read/action status.
38. Donors can view their full notification history in their dashboard.

3.5 Dashboards

3.5.1 Donor Dashboard

- Profile summary: blood type, donation count, trust badge, availability status.
- Incoming requests: active notifications with Accept/Decline buttons.
- Donation history: log of past accepted requests and confirmed donations.
- Availability toggle: quickly mark self as unavailable.

3.5.2 Hospital Dashboard

- Create new blood request (button always visible).
- Active requests: live list with real-time donor response counters.
- Fulfilled/Expired requests: historical view.
- Donor responses: for each request, list of donors who accepted with contact info.

3.5.3 Admin Panel

- Pending hospital approvals with Approve / Reject actions.
- User management: view, suspend, or delete donors and hospitals.
- Platform statistics: total users, requests, fulfillment rate, active donors by city.
- System health: WebSocket connection status, notification delivery rates.

3.6 Trust & Verification System

3.6.1 Donor Trust Badges

Badge	Requirement	Benefit
New Donor	Account created & verified	Access to the platform
Active Donor	1+ confirmed donations	Visible badge on profile
Trusted Donor	3+ confirmed donations	Priority in matching sort order
Hero Donor	10+ confirmed donations	Top priority + special badge

39. Hospital accounts must be approved by Admin before any access is granted.
40. Donors can optionally upload a national ID for verified identity status.
41. Donation confirmations can be submitted by donors and validated by hospitals.

4. Non-Functional Requirements

4.1 Performance

- Notification delivery: matched donors must receive WebSocket push within 3 seconds of request creation.
- Page load time: all dashboard pages must load within 2 seconds on a standard connection.
- The matching engine must complete and notify all donors within 5 seconds for up to 500 matched donors.
- Database queries for matching must be indexed on blood_type, city, district, and status.

4.2 Security

- All passwords are hashed using bcrypt (Laravel default).
- CSRF protection enabled on all forms (Laravel default).
- Hospital accounts require Admin approval — no self-serve access to request creation.
- Donor contact details are only visible to hospitals after a donor explicitly accepts a request.
- All API routes are protected by Laravel Sanctum authentication middleware.
- WebSocket channels are authenticated — donors only receive notifications on their private channel.

4.3 Usability

- Mobile-first design: all pages must be fully usable on a 375px wide screen.
- The PWA must be installable on Android and iOS devices.
- Critical actions (Accept request, Create request) must require no more than 3 taps from the dashboard.
- Arabic RTL layout support should be planned from the start, even if launched in English first.

4.4 Reliability & Availability

- The platform must handle WebSocket reconnections gracefully — if a connection drops, Laravel Echo automatically attempts to reconnect without data loss.
- Blood requests that fail to notify donors (e.g. server restart) must have a recovery mechanism — a queue job that re-checks and re-notifies on restart.
- Target uptime: 99.5% (allows ~3.6 hours downtime per month).

4.5 Scalability

- The system is designed for city-based matching — no GPS computation required, enabling fast SQL queries.
- Laravel queues (database driver for development, Redis for production) handle notification dispatching asynchronously.
- Reverb is self-hosted and can be scaled horizontally if needed.

5. Data Requirements

5.1 Core Entities

The following entities form the core of the Wareed data model. Full ERD will be produced in the Design phase.

5.1.1 Users (Donors)

Field	Type	Notes
id	bigint PK	Auto-increment
name	varchar(100)	Full name
email	varchar(150)	Unique, used for login
password	varchar(255)	Bcrypt hashed
phone	varchar(20)	Contact number
blood_type	enum	A+, A−, B+, B−, AB+, AB−, O+, O−
city	varchar(100)	Used for matching
district	varchar(100)	Used for proximity prioritization
status	enum	available, unavailable, on_cooldown
last_donation_date	date	Null if never donated
donation_count	int	Total confirmed donations
email_verified_at	timestamp	Null until verified
created_at / updated_at	timestamp	Auto-managed by Laravel

5.1.2 Hospitals

Field	Type	Notes
id	bigint PK	Auto-increment
name	varchar(150)	Hospital official name
email	varchar(150)	Unique, used for login
password	varchar(255)	Bcrypt hashed
phone	varchar(20)	Main contact number
city	varchar(100)	Location for matching

district	varchar(100)	Sub-location
license_number	varchar(100)	Official registration number
status	enum	pending, approved, rejected, suspended
approved_at	timestamp	When Admin approved the account
created_at / updated_at	timestamp	Auto-managed

5.1.3 Blood Requests

Field	Type	Notes
id	bigint PK	Auto-increment
hospital_id	bigint FK	References hospitals.id
blood_type	enum	The blood type required
units_needed	int	Number of donation units required
urgency	enum	critical, urgent, standard
status	enum	active, partially_fulfilled, fulfilled, expired, cancelled
contact_person	varchar(100)	Name of hospital contact
contact_phone	varchar(20)	Direct contact number
notes	text	Optional additional information
expires_at	timestamp	Auto-set based on urgency level
created_at / updated_at	timestamp	Auto-managed

5.1.4 Donor Responses

Field	Type	Notes
id	bigint PK	Auto-increment
blood_request_id	bigint FK	References blood_requests.id
donor_id	bigint FK	References users.id
status	enum	notified, accepted, declined
responded_at	timestamp	When donor took action
donation_confirmed	boolean	Hospital confirmed donation occurred

created_at / updated_at	timestamp	Auto-managed

5.1.5 Notifications

Field	Type	Notes
id	bigint PK	Auto-increment
user_id	bigint FK	Target donor
type	varchar(100)	Laravel notification class name
data	json	Notification payload (request details)
read_at	timestamp	Null until donor reads it
created_at	timestamp	Auto-managed

6. Use Case Summary

ID	Use Case	Actor	Priority
UC-01	Register as Donor	Guest	Must Have
UC-02	Register as Hospital	Guest	Must Have
UC-03	Login to Platform	All Users	Must Have
UC-04	Admin Approves Hospital	Admin	Must Have
UC-05	Hospital Creates Blood Request	Hospital	Must Have
UC-06	System Matches Donors	System	Must Have
UC-07	System Notifies Matched Donors	System	Must Have
UC-08	Donor Accepts Request	Donor	Must Have
UC-09	Donor Declines Request	Donor	Must Have
UC-10	Hospital Views Donor Responses	Hospital	Must Have
UC-11	Hospital Marks Request Fulfilled	Hospital	Must Have
UC-12	System Expires Old Requests	System (cron)	Must Have
UC-13	Donor Updates Availability	Donor	Should Have
UC-14	Donor Views Donation History	Donor	Should Have
UC-15	Hospital Cancels Request	Hospital	Should Have
UC-16	Admin Suspends User	Admin	Should Have
UC-17	Admin Views Platform Stats	Admin	Should Have
UC-18	Install Platform as PWA	Donor / Hospital	Should Have
UC-19	Receive Background Push Notification	Donor	Should Have
UC-20	Donor Earns Trust Badge	System	Nice to Have

7. Constraints & Assumptions

7.1 Constraints

- The platform is built with Laravel 11 — no other backend framework should be used.
- Real-time functionality is exclusively powered by Laravel Reverb — no Pusher or Firebase.
- Location matching is city/district-based — no GPS or geolocation APIs are required.
- The platform is web-based; native iOS/Android apps are out of scope for version 1.
- All hosting is self-managed (VPS) — no serverless or managed WebSocket services.

7.2 Assumptions

- Hospitals have reliable internet access to use the web dashboard.
- Donors primarily access the platform via mobile browsers (hence PWA focus).
- The initial launch targets Egyptian cities — city/district data will be a static list, not a third-party API.
- Blood type compatibility follows standard medical guidelines (ABO + Rh factor only).
- Donation cooldown is set to 56 days (standard whole blood donation recovery period).
- Email is the primary communication channel for account events; push notifications via WebSocket for urgent alerts.

7.3 Out of Scope (Version 1)

- GPS/map-based donor location tracking.
- Native mobile applications (iOS/Android).
- SMS notifications (can be added in v2 via Vonage/Twilio).
- Platelet or plasma donation matching (whole blood only for v1).
- Payment or compensation system for donors.
- Integration with hospital Electronic Health Record (EHR) systems.

8. Development Roadmap

The project is divided into 5 milestones. Each milestone builds on the previous and produces a working, testable piece of the system.

Phase	Milestone	Key Deliverables
Phase 1	Foundation	Laravel project setup, authentication (Donor + Hospital + Admin), email verification, database migrations for all core tables
Phase 2	Hospital Flow	Hospital dashboard, create/manage blood requests, request lifecycle management, Admin approval system
Phase 3	Matching Engine	Blood type compatibility logic, city/district matching query, donor eligibility checks, cooldown management
Phase 4	Real-Time System	Laravel Reverb setup, private WebSocket channels, Laravel Echo frontend integration, live dashboard updates, notification storage
Phase 5	PWA + Polish	Web App Manifest, Service Worker, install prompts, trust badge system, Admin stats panel, UI/UX refinement

Next Step: System Design

Now that requirements are defined, the next phase is System Design, which includes:

- Full Entity Relationship Diagram (ERD) with all tables and relationships
- System Architecture Diagram (Laravel + Reverb + Echo flow)
- Real-Time Event Flow Diagram (request creation → matching → notification)
- Route Map (all web and API routes)
- Database Schema with indexes and constraints

9. Appendix

9.1 Blood Type Compatibility Matrix

This matrix defines which donor blood types can donate to which recipient blood types. The matching engine uses this logic.

Donor \ Recipient	O−	O+	A−	A+	B−	B+	AB−	AB+
O−	✓	✓	✓	✓	✓	✓	✓	✓
O+	—	✓	—	✓	—	✓	—	✓
A−	—	—	✓	✓	—	—	✓	✓
A+	—	—	—	✓	—	—	—	✓
B−	—	—	—	—	✓	✓	✓	✓
B+	—	—	—	—	—	✓	—	✓
AB−	—	—	—	—	—	—	✓	✓
AB+	—	—	—	—	—	—	—	✓

9.2 Urgency Level Configuration

Urgency	Label	Auto-Expire After	Notification Priority
critical	Critical	6 hours	Highest — immediate
urgent	Urgent	24 hours	High
standard	Standard	72 hours	Normal

End of Wareed SRS v1.0